Laboratory 0b110

Expected submission ON PORTALE DELLA DIDATTICA: SURNAME_NAME_MATRICOLA_lab_06.zip

Including (in case you have projects with the same name, you can prepend lab_ to them):
- program_1, 3 and 4.
- lab_06.pdf (complete this file and export it as PDF)

Submission deadline: 10/11/2026

This lab will start to delve into ARM programming.

Getting started with Keil µVision

Installation

Download MDK-Arm from https://www.keil.com/download/product/. During the download you are asked for your e-mail address and affiliation: use your institutional account (<name.surname>@studenti.polito.it) and "Politecnico di Torino" as affiliation. Without a license the tool runs as a Lite/Evaluation edition with a few limitations. Once installed, launch µVision from its icon.

Follow the guide as instructed at https://cas.polito.it/tools/keil-uvision/: download the Device Family Pack for the NXP LPC1768 and install it (follow the website instruction).

Opening and running the template project

- Take the 00_ASM_template project from the course material (ARM → examples folder). Since the project includes quite a lot of files, copy it into a separate folder for each program (e.g., program_1, program_3, program_4).
- Open the project by double-clicking the ASM_template.uvproj file (or Project → Open Project… in µVision).
- Legacy pack: the LPC device environment (NXP LPC1768) is not installed by default; dialogs will appear and guide you to the website to download and install it.
- In the Project window expand Target 1 → Source Group 1 and open startup.s. The Reset_Handler does nothing but bringing the execution to your code: write your instructions at the "; your code here" comment, between LDR R0, =Reset_Handler and BX R0.
- Build your code with the Build button of the toolbar (F7); Compile translates only the current file, Rebuild recompiles all files. Check the Build Output window for errors and warnings.
- Debug setup: click Options for Target (the "magic stick" icon) → Debug tab and select Use Simulator (software debug, emulated functionalities). The right-hand option (Use: ULINK2/ME Cortex Debugger) is for hardware debug on the board.
- Start/stop the debug session with the Start/Stop Debug Session button (magnifying-glass icon, Ctrl+F5). A yellow pointer indicates the code line that is going to be executed.

Main debugger tools

Execution control: RST resets the device; Run (F5) executes the program and stops only at a breakpoint; Step Into (F11) executes one instruction; Step Over (F10) executes a subroutine call as a single step; Run to Cursor (Ctrl+F10) stops at the line where the cursor is. The status bar (t1) shows the simulated execution time.

Registers window: the Core group shows R0–R12, R13 (SP), R14 (LR), R15 (PC) and xPSR (expand it to see the N, Z, C, V flags); the Internal group shows the processor state (Mode Thread/Handler, Privilege, Stack MSP/PSP). Values changed by the last step are highlighted.

Breakpoints: double-click the grey area close to a code line (or press F9) to set/remove a breakpoint, shown as a red dot. Debug → Breakpoints… (Ctrl+B) lists and manages all breakpoints.

Peripherals and memory: the Peripherals menu opens the windows of the device modules (e.g., Clocking & Power Control → Clock Generation Schematic); the Memory window shows the content of memory at a given address.

Warm-up exercises (Keil µVision)

Exercise 1: Viewing registers. Starting from the 00_ASM_template project, write the following code in the Reset_Handler of startup.s (at "; your code here"), build it, select the simulator and start the debug session:

```
MOV  R0, #10
MOV  R1, #0xFF
ADD  R2, R0, R1
```

Execute it one instruction at a time (F11) while watching the Registers window. Report the value of R2 in hexadecimal and decimal, and the value of PC before and after each step. By how many bytes does PC increase at each instruction? Then double-click on R0 in the Registers window, change its value, and re-execute the ADD (e.g., with Reset and a new run): does R2 change accordingly?

Instruction	R0 (hex)	R1 (hex)	R2 (hex)	PC before	PC after
`MOV  R0, #10`					
`MOV  R1, #0xFF`					
`ADD  R2, R0, R1`					

Answer:

Exercise 2: Reporting the processor state. Execute the following instructions step by step and, after each one marked with S (flag-setting), report the result and the N, Z, C, V flags shown in xPSR. Also report the processor state shown in the Internal group of the Registers window (Mode, Privilege, Stack):

```
MOV  R0, #5
SUBS R1, R0, #5          ; result zero
MOVS R2, #0
SUBS R3, R2, #1          ; result negative
LDR  R4, =0x7FFFFFFF
ADDS R5, R4, #1          ; signed overflow
```

Instruction	Result (hex)	N	Z	C	V
`SUBS R1, R0, #5`					
`MOVS R2, #0`					
`SUBS R3, R2, #1`					
`ADDS R5, R4, #1`					

Answer:

Exercise 3: Setting breakpoints. Write the following loop:

```
      MOV  R0, #0
      MOV  R1, #5
loop  ADD  R0, R0, R1
      SUBS R1, R1, #1
      BNE  loop
stop  B    stop
```

Set a breakpoint (double-click the grey area next to the line) on the ADD instruction and run the program (F5). Each time execution stops, note the values of R0 and R1: how many times is the breakpoint hit? Then remove it, set a breakpoint on the stop line, press Reset and run again: report the final value of R0 and the execution time (t1) shown in the status bar. Finally, open Debug → Breakpoints… (Ctrl+B) and check the list of active breakpoints.

Answer:

Exercise 4: Writing your first program. Starting from a new copy of the 00_ASM_template project, write a program using the ARM assembly that performs the following operations:

1. Initialize registers R3, R4, and R5 with arbitrary (chosen by you) signed values, at least one of them negative.
2. Subtract R3 from R4 (R4 – R3) and store the result in R6.
3. Sum R5 to R6 (R5 + R6) and store the result in R7.
4. Build the program and run it in the simulator. Report the code size, given by the "Program Size: Code=…" line in the Build Output window, and the execution time, shown as t1 in the status bar when execution reaches the end of your code (e.g., with a breakpoint on the last instruction).

Code size (bytes)	**Execution time (t1)**